

Assignment – 18.3

2303A510i9

Task 1 – Public Transport Route Fare API

PROMPT : generate a Public Transport API to retrieve fare details between two stations.

CODE AND OUTPUT :

```
lab 18.3.py > ...
1  # generate a Public Transport API to retrieve fare details between two stations.
2  class PublicTransportAPI:
3      def __init__(self):
4          # Sample fare data between stations
5          self.fare_data = {
6              ('Station A', 'Station B'): 2.5,
7              ('Station A', 'Station C'): 3.0,
8              ('Station B', 'Station C'): 1.5,
9              ('Station B', 'Station D'): 2.0,
10             ('Station C', 'Station D'): 2.5,
11         }
12
13     def get_fare(self, station_from, station_to):
14         # Retrieve fare details between two stations
15         fare = self.fare_data.get((station_from, station_to))
16         if fare is not None:
17             return f"The fare from {station_from} to {station_to} is ${fare:.2f}."
18         else:
19             return f"No fare information available for the route from {station_from} to {station_to}."
20
21 # Example usage
22 api = PublicTransportAPI()
23 print(api.get_fare('Station A', 'Station B'))
24 print(api.get_fare('Station A', 'Station D'))
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

▼ TERMINAL

```
PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & 'c:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding\code\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '58593' '--' 'c:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding\code\lab 18.3.py'
The fare from Station A to Station B is $2.50.
No fare information available for the route from Station A to Station D.
PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & "c:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding\code\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher" '58593' '--' 'c:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding\code\lab 18.3.py'
```

JUSTIFICATION :

Integrating a fare API ensures commuters get accurate cost information between stations. Input validation and error handling make the system reliable even during downtime. A structured output helps users quickly interpret and apply fare details.

Task 2 – Currency Exchange Rates

PROMPT :

generate a Currency Exchange API to convert INR to USD, EUR, and GBP.

CODE AND OUTPUT :

```

25 #generate a CurrencyExchange API to convert INR to USD,EUR, and GBP.
26 class CurrencyExchangeAPI:
27     def __init__(self):
28         # Sample exchange rates from INR to other currencies
29         self.exchange_rates = {
30             'USD': 0.013, # 1 INR = 0.013 USD
31             'EUR': 0.011, # 1 INR = 0.011 EUR
32             'GBP': 0.009, # 1 INR = 0.009 GBP
33         }
34     def convert_inr(self, amount_in_inr, target_currency):
35         # Convert INR to the target currency
36         rate = self.exchange_rates.get(target_currency)
37         if rate is not None:
38             converted_amount = amount_in_inr * rate
39             return f"{amount_in_inr} INR is approximately {converted_amount:.2f} {target_currency}."
40         else:
41             return f"Currency '{target_currency}' not supported for conversion."
42 # Example usage
43 exchange_api = CurrencyExchangeAPI()
44 print(exchange_api.convert_inr(1000, 'USD'))
45 print(exchange_api.convert_inr(1000, 'EUR'))
46 print(exchange_api.convert_inr(1000, 'GBP'))
47 print("\n" + "=" * 80)
48 print("TASK 4: Dictionary Key Error")
49 print("=" * 80)

```

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

AZURE

▼

TERMINAL

```

● PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & 'c:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding\code\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61326' '--' 'c:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding\code\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher'
1000 INR is approximately 13.00 USD.
1000 INR is approximately 11.00 EUR.
1000 INR is approximately 9.00 GBP.

=====
TASK 4: Dictionary Key Error
=====

```

JUSTIFICATION :

Currency conversion supports global transactions and travel planning by providing real-time exchange values. Validating inputs and handling API errors ensures dependable financial results. Clear tabular output makes comparisons across currencies easy to understand.

Task 3 – GitHub Repository Info Fetcher

PROMPT :

GitHub Repository Info Fetcher

CODE AND OUTPUT :

```
1 # GitHub Repository Info Fetcher
2 import requests
3 def fetch_github_repo_info(owner, repo):
4     url = f"https://api.github.com/repos/{owner}/{repo}"
5     response = requests.get(url)
6
7     if response.status_code == 200:
8         repo_info = response.json()
9         print(f"Repository Name: {repo_info['name']}")
10        print(f>Description: {repo_info['description']}")
11        print(f"Stars: {repo_info['stargazers_count']}")
12        print(f"Forks: {repo_info['forks_count']}")
13    else:
14        print(f"Failed to fetch repository info. Status code: {response.status_code}")
15 # Example usage
16 fetch_github_repo_info("octocat", "Hello-World")
17
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

▼ TERMINAL

```
● PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & 'c:\Users\Ramya Sri\OneDrive\New folder\
e' 'c:\Users\Ramya Sri\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\laun
ive\New folder\AI Assistant coding\lab 18.3.py'
Repository Name: Hello-World
Description: My first repository on GitHub!
Stars: 3550
Forks: 5948
● PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & "c:\Users\Ramya Sri\OneDrive\New folder\
s1"
```

JUSTIFICATION :

Fetching repository data gives developers insights into project activity and popularity. Robust error handling keeps the tool resilient against invalid inputs or rate limits. Displaying stars, forks, and issues allows quick evaluation of a project's health.

Task 4 – Real-Time Application: News Headlines Aggregator

PROMPT :

Build a News Aggregator using a free News API.

CODE AND OUTPUT :

```

1 # News Headlines Aggregator (working robust version)
2 import requests
3 import time
4 # Replace with your API key from https://newsapi.org
5 API_KEY = "YOUR_API_KEY"
6 BASE_URL = "https://newsapi.org/v2/top-headlines"
7 VALID_CATEGORIES = ["sports", "technology", "health"]
8 mock_news = {
9     "sports": [
10         "India wins cricket match",
11         "Football finals scheduled",
12         "Olympics updates released",
13         "Tennis championship highlights",
14         "New athletics record set"
15     ],
16     "technology": [
17         "New AI model launched",
18         "Latest smartphone released",
19         "Cybersecurity trends rising",
20         "Tech companies earnings report",
21         "Breakthrough in quantum computing"
22     ],
23     "health": [
24         "Healthy lifestyle tips",
25         "New vaccine developments",
26         "Mental health awareness programs",
27         "Nutrition research updates",
28         "Fitness trends increasing"
29     ]
30 }
31
32 def fetch_news(category):
33     url = f"{BASE_URL}?category={category}&country=in&apiKey={API_KEY}"
34     return requests.get(url, timeout=10)
35
36 def get_news(category):
37     for attempt in range(3):
38         try:
39             response = fetch_news(category)
40             if response.status_code == 200:
41                 data = response.json()
42                 if data.get("status") != "ok":
43                     raise Exception("Invalid API response")
44                 articles = data.get("articles", [])
45                 if articles:
46                     return [article["title"] for article in articles[:5]]
47                 else:
48                     raise Exception("No articles found")
49             elif response.status_code == 401:
50                 raise PermissionError("Invalid or missing API key.")
51             elif response.status_code == 400:
52                 raise ValueError("Invalid category.")
53             elif response.status_code == 429:
54                 raise Exception("Too many requests (rate limit).")
55             else:
56                 raise Exception(f"HTTP Error: {response.status_code}")
57         except (requests.exceptions.Timeout, requests.exceptions.ConnectionError) as exc:
58             print(f"Attempt {attempt + 1} failed: {exc}")
59             if attempt < 2:
60                 print("Retrying...\n")
61                 time.sleep(2)
62             else:
63                 print("Using fallback data.\n")
64                 return mock_news.get(category, [])
65         except Exception as e:
66             print(f"Attempt {attempt + 1} failed: {e}")
67             if attempt < 2:
68                 print("Retrying...\n")
69                 time.sleep(2)

```

by clicking to restart.

```

ate.ps1"
(.venv) PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & "c:/Users/Ramya Sri/OneD
ts/python.exe" "c:/Users/Ramya Sri/OneDrive/New folder/AI Assistant coding/lab 18.3.py"
=== News Headlines Aggregator ===
Enter category (sports/technology/health): sports
Attempt 1 failed: Invalid or missing API key.
Retrying...

Attempt 2 failed: Invalid or missing API key.
Retrying...

Attempt 3 failed: Invalid or missing API key.
Using fallback data.

Top 5 Headlines:
1. India wins cricket match
2. Football finals scheduled
3. Olympics updates released
4. Tennis championship highlights
5. New athletics record set

```

JUSTIFICATION :

A news aggregator delivers timely headlines in categories like sports or technology. Error handling and retry logic ensure users still get results despite failures. Numbered lists make the headlines easy to read and engaging.

Task 5 – COVID-19 Statistics API Integration

PROMPT : #Connect to a COVID-19 Statistics API to fetch country-wise COVID data.india

CODE AND OUTPUT :

```
lab 18.3.py > get_covid_data
1 #Connect to a COVID-19 Statistics API to fetch country-wise COVID data.India
2 import requests
3 # API URL (disease.sh free API)
4 BASE_URL = "https://disease.sh/v3/covid-19/countries/"
5 def get_covid_data(country):
6     try:
7         url = BASE_URL + country
8         response = requests.get(url)
9         # Handle response
10        if response.status_code == 200:
11            return response.json()
12        elif response.status_code == 404:
13            print("Country not found. Please check the name.")
14            return None
15        else:
16            print(f"Error: {response.status_code}")
17            return None
18    except requests.exceptions.RequestException:
19        print("Network error. Please check your Internet connection.")
20        return None
21
22    try:
23        print("=== COVID-19 Statistics ===")
24        country = input("Enter country name: ").strip()
25        # Input validation
26        if not country:
27            raise ValueError("Country name cannot be empty.")
28        data = get_covid_data(country)
29        if data:
30            print("\nCOVID-19 Statistics")
31            print("-----")
32            print(f"Country           : {data.get('country')}")
33            print(f"Total Cases       : {data.get('cases')}")
34            print(f"Total Deaths     : {data.get('deaths')}")
35            print(f"Total Recovered   : {data.get('recovered')}")
36            print("-----")
37        except ValueError as ve:
38            print(f"\nInput Error: {ve}")
39        except Exception as e:
40            print(f"\nUnexpected Error: {e}")
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

▼ TERMINAL

```
er\AI Assistant coding\venv\Scripts\python.exe' 'c:\Users\Ramya Sri\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50069' '--' 'c:\Users\Ramya Sri\O
=== COVID-19 Statistics ===
Enter country name: india

COVID-19 Statistics
-----
Country           : India
Total Cases       : 45035393
Total Deaths     : 533570
Total Recovered   : 0
-----

(.venv) PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding>
```

JUSTIFICATION :

Country-wise COVID statistics are vital for awareness and decision-making. Error handling and retry logic guarantee accurate data retrieval despite API limits. Presenting confirmed, recovered, deaths, and active cases provides a complete health snapshot.